

BBH Graphs Process Documentation

Zoey Zhu (zyz2000)

June 2026

Contents

1	Useful Links	3
2	Environment Setup	3
3	Imports	4
4	Setting Up TARGET_NSIDE and eROSITA Survey Windows	5
5	Fetching All Events from GWTC	5
6	Fetching Skymaps	6
7	Plotting Skymaps	6
8	Computing Scattering and Absorption Optical Depths	7
9	Graphing Scattering and Absorption Probabilities	7
10	Overlaying Skymaps on Probability Maps	7
11	Calculating Percent Overlap	8
12	Calculating Percent of Sky Each Skymap Takes Up	8
13	Calculating Probability of Total Failure and At Least One Success	8
14	eROSITA Scan Timing	9
14.1	Physical Basis	9
14.2	Why Ecliptic Coordinates?	9
14.3	get_sun_lambdas (Cell 10)	9
14.4	get_observation_time_eRosita (Cell 11)	9
14.5	get_observation_time_eRosita_ec (Cell 16.1)	9
14.6	get_gw_time (Cell 12)	10
14.7	Time Delay Computation	10
14.8	Fast Batch Timing (Cell 16.2)	10
14.9	Filtering (Cell 17)	10

15 Validation: MAXI J1348–630 (Cell 19)	10
16 Graphing the SFD Dust Map (Optional, Cell 5)	11

Useful Links

GWOSC Catalogs

- GWTC full catalog (HTML):
<https://gwosc.org/eventapi/html/GWTC/>
- GWTC full catalog (JSON):
<https://gwosc.org/api/v2/catalogs/GWTC/events?format=api&include-default-parameters=true>
- GWTC-4.0 cumulative (JSON):
<https://gwosc.org/api/v2/event-versions?release=GWTC-1-confident%2CGWTC-2.1-confident%2CGWTC-3-confident%2CGWTC-4.0&lastver=true&include-default-parameters=true&format=api>
- GWTC-2.1 cumulative (JSON):
<https://gwosc.org/api/v2/event-versions?release=GWTC-1-confident%2CGWTC-2.1-confident&lastver=true&include-default-parameters=true&format=api>
- GWTC-1.0 cumulative (JSON):
<https://gwosc.org/api/v2/event-versions?release=GWTC-1-confident&lastver=true&include-default-parameters=true&format=api>

GWTC Zenodo Skymaps

- GWTC-4 v2: <https://zenodo.org/records/17602505>
- GWTC-3 v2: <https://zenodo.org/records/8177023>
- GWTC-2.1 v2: <https://zenodo.org/records/6513631> (GWTC-1 subsumed in this)

HI4PI Column Densities

https://lambda.gsfc.nasa.gov/product/foreground/fg_hi4pi_get.html

eROSITA

- eROSITA survey operations and scan dates (Coutinho et al. 2022):
<https://doi.org/10.1117/12.2628946>

Environment Setup

Replace [...] with your `username@hostname`.

Listing 1: Check prerequisites

```
(base) [...] ~ % conda --version      # tested on conda 25.11.0
(base) [...] ~ % python3 --version    # tested on Python 3.13.9
(base) [...] ~ % git --version         # tested on git 2.39.5 (Apple Git-154)
# VSCode: Version 1.121.0 (Universal)
```

Listing 2: Create and activate conda environment (Python 3.11 for healpy compatibility; run inside the project folder)

```
conda create -n bbh-visualization python=3.11
conda activate bbh-visualization
# Prompt now shows: (bbh-visualization) [...] BBH Visualization %
```

Listing 3: Install packages

```
pip install healpy astropy dustmaps matplotlib requests jupyter numpy pandas ligo.skymap
```

Listing 4: Link to GitHub

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
git init
git remote add origin https://github.com/[username]/[repo]
git pull origin main
```

Listing 5: Create notebook file

```
touch bbh_visualization.ipynb
```

Create `.gitignore` with the following entries:

```
*.fits
*.fits.gz
dustmaps/
.venv/
env/
__pycache__/_
*.pyc
*.pyo
.ipynb_checkpoints/
.DS_Store
*.hdf5
*.h5
```

Add the repo to GitHub Desktop and do an initial commit. In VSCode, select `bbh-visualization` as the kernel. Every time you work on the project, activate the environment first:

```
conda activate bbh-visualization
```

Imports

Listing 6: All imports (Cell 1.1)

```
import numpy as np
import healpy as hp
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from matplotlib.patches import Patch, Annulus
import requests
import os
import time
import tarfile
```

```

from astropy.io import fits
from ligo.skymap.io import read_sky_map
import dustmaps.sfd
from dustmaps.sfd import SFDQuery
from astropy.coordinates import SkyCoord, GeocentricMeanEcliptic, get_body
import astropy.units as u
from astropy.time import Time
import csv
import math
import pandas as pd

```

Key additions since initial setup:

- **Annulus** — draws halo rings in Cell 19
- **GeocentricMeanEcliptic, get_body** — ecliptic coordinate frame and Sun position calculations for the eROSITA scan model
- **Time** — GW trigger time parsing, eROSITA survey window definitions, and delay arithmetic
- **pandas** — reads and filters eROSITA timing CSVs in Cell 17

Setting Up TARGET_NSIDE and eROSITA Survey Windows

TARGET_NSIDE = 1024: HEALPix divides the sphere into $12 \times \text{NSIDE}^2$ equal-area pixels. $\text{NSIDE} = 1024$ matches the native resolution of HI4PI so no information is lost. Everything is resampled to this resolution to make pixel-by-pixel comparisons possible.

eROSITA survey windows (dates from Coutinho et al. 2022), stored as `astropy.time.Time` constants in Cell 1.2:

Survey	Start (UTC)	End (UTC)
eRASS1	2019-12-13	2020-06-11
eRASS2	2020-06-11	2020-12-15
eRASS3	2020-12-15	2021-06-16
eRASS4	2021-06-16	2021-12-19

These constants are used in all eROSITA timing calculations (Cells 10–18).

Fetching All Events from GWTC

Use the `requests` library to make paginated GET requests to the GWOSC REST API with a 0.5-second delay between pages. Filter events for BBH by checking that both `mass_1_source` and `mass_2_source` exceed $3 M_{\odot}$ (the neutron star / black hole boundary). Return 0 if a mass field is missing so the filter does not crash.

A second API query restricted to GWTC-1-confident through GWTC-4.0 isolates the 167 BBH events with published skymaps. Skymap URLs are embedded in the API response JSON — the structure varies between catalog versions, so click through the JSON endpoints to find where the links live.

Fetching Skymaps

- GWTC-1-confident has been subsumed into GWTC-2.1; do not use old LIGO DCC links.
- GWTC-2.1 and GWTC-3 skymaps are split across multiple tar files. GWTC-4.0 has one large tar file.

Process:

1. Collect unique tar URLs into a dictionary: `{tar_url: [event_names]}`.
2. Create a `skymaps/` directory.
3. Stream each tar to a temp file in 8 MB chunks (not into RAM) to keep memory usage constant and avoid crashes on large archives.
4. Use `tarfile` to extract only the `.fits` or `.fits.gz` member matching the target event name.
5. Delete the temp file.
6. Retry logic: up to 3 attempts with exponential back-off (1s, 2s, 4s). Already-extracted files are skipped, so re-running is safe.

Plotting Skymaps

Use `imshow` rather than `scatter` — each displayed pixel matches the HEALPix pixel size. Use `matplotlib` instead of `healpy`'s built-in viewers for more flexibility.

Pipeline for each file:

1. Read with `ligo.skymap.read_sky_map(file, nest=True)` → flat probability array in NESTED ordering.
2. Convert to RING ordering: `hp.reorder(skymap, n2r=True)`.
3. Resample to `TARGET_NSIDE` with `hp.ud_grade`; renormalize (`/= sum()`) afterward.
4. Compute 90% credible region: sort pixels highest → lowest with `np.argsort[::-1]`, cumulative sum with `np.cumsum`, find threshold index with `searchsorted(0.90)`, build binary confidence map.
5. Build 1400×700 equirectangular grid, convert to HEALPix (θ, ϕ) , look up confidence values, plot with `ax.imshow`.

Display notes:

- The 1400×700 display grid has far fewer pixels than the 12 million HEALPix pixels — this is fine because overlap calculations use HEALPix directly.
- Multi-event plots: vary hue only; reserve grays for the probability background.
- Figure layout: 14×9 with `subplots_adjust(top=7/9)` gives a 14×7 plot area and space for the legend below.
- Save with `plt.savefig`.

Computing Scattering and Absorption Optical Depths

Scattering. The single-grain cross section in the small-angle forward-scattering limit scales as

$$\sigma \propto a^2 \cdot \left(\frac{2\pi a}{\lambda} \right)^2 = \frac{a^4}{\lambda^2} \propto a^4 E^{-2}$$

since $\lambda = hc/E \propto E^{-1}$ for X-rays. Integrating over the Mathis, Rumpl & Nordsieck (1977) power-law grain size distribution ($dn/da \propto a^{-3.5}$, $a_{\min} = 0.005 \mu\text{m}$, $a_{\max} = 0.3 \mu\text{m}$) gives:

$$\tau_{\text{scattering}} = 1.17 \times 10^{-22} \cdot N_H \cdot \frac{a_{\max}}{0.3 \mu\text{m}} \cdot E_{\text{keV}}^{-2}$$

Absorption. Using tabulated photoelectric cross sections from Morrison & McCammon (1983):

$$\tau_{\text{absorption}} = \sigma_{\text{phot,ISM}}(E) \cdot N_H$$

where $\sigma_{\text{phot,ISM}}$ is looked up per energy (1.0–3.0 keV) and already incorporates E^{-3} scaling.

`get_tau_scattering_and_absorption(E_keV)` loads HI4PI from `NHI_HPX.fits`, clips negative column density values to 0, and returns both tau arrays indexed by galactic HEALPix pixel.

Graphing Scattering and Absorption Probabilities

Uses `imshow`. HI4PI is in galactic coordinates; plots are in equatorial — coordinate conversion is required at every pixel.

Per-pixel pipeline:

1. Build 1400×700 equirectangular (RA, Dec) grid.
2. Flatten to 1D, convert to galactic (l, b) with `SkyCoord`, reshape to 2D.
3. Convert galactic $(l, b) \rightarrow$ HEALPix (θ, ϕ) , look up pixel indices, retrieve probability values.
4. Compute $P = 1 - e^{-\tau}$.
5. Assign grayscale by threshold: darker = higher scattering or lower absorption.
6. Save to `x-rays E = {E}keV/` folder.

Three map types are produced: scattering only, absorption only, and a combined mask where scattering > 0.1 **and** absorption < 0.95 (the absorption threshold corresponds to $\tau_{\text{absorption}} = 3$).

Overlaying Skymaps on Probability Maps

Uses `imshow` for both layers. Events are plotted in chronological chunks of 10.

1. Compute the gray probability background once per photon energy.
2. For each chunk: create a new figure, draw the gray background first, then draw each event's 90% credible region as a colored RGBA layer (transparent outside the region).
3. Add a legend below the plot area.
4. Save as PNG before moving to the next chunk.

Calculating Percent Overlap

Coordinate alignment problem. HI4PI uses galactic HEALPix pixel indices; GW skymaps use equatorial. At the same NSIDE, pixel index i refers to different sky patches in each system. Fix by remapping the probability arrays to equatorial ordering before comparison:

equatorial pixel $i \xrightarrow{\text{hp.pix2ang}} (\text{RA}, \text{Dec}) \xrightarrow{\text{SkyCoord}} \text{galactic } (l, b) \xrightarrow{\text{hp.ang2pix}} \text{galactic pixel } j \xrightarrow{\text{lookup}} \text{probability_eq}[i]$

After remapping, `confidence_map[i]` and `probability_mask[i]` refer to the same patch of sky and can be combined with a bitwise AND.

Calculation:

```
overlap_pixels      = np.sum(confidence_map & probability_mask)
skymap_pixels      = np.sum(confidence_map)
overlap_percentage  = 100 * overlap_pixels / skymap_pixels
```

Results are collected in a dictionary `{event_name: overlap_%}`, sorted descending into `greatest_overlaps` (a list of tuples), and saved to CSV.

Calculating Percent of Sky Each Skymap Takes Up

Done in the same cell as the overlap calculation:

```
area_percentage = np.sum(confidence_map) / TOTAL_PIXELS
```

Stored in `skymap_area_percentages_dict`, reordered to match the `greatest_overlaps` rank order, and saved to a separate CSV.

Calculating Probability of Total Failure and At Least One Success

Direct multiplication of 167 small probabilities underflows to 0.0 in float64, so log space is used:

```
log_prob_total_failure = 0.0
for _, pct in greatest_overlaps[:n]:
    log_prob_total_failure += math.log(max(1 - pct/100, 1e-10))
    # Clamp to 1e-10 to handle the 100% overlap edge case

prob_total_failure      = math.exp(log_prob_total_failure)
prob_at_least_one_success = -np.expm1(log_prob_total_failure)
# np.expm1 computes e^x - 1 with full precision near x=0, avoiding cancellation
```

Treating each BBH event as an independent Bernoulli trial with success probability $p_i = \text{overlap}_i/100$:

$$P(\text{total failure}) = \prod_i (1 - p_i), \quad P(\text{at least one success}) = 1 - \prod_i (1 - p_i)$$

Can be run on any subset (all 167, top half, top quarter, etc.). Results are appended to a CSV.

eROSITA Scan Timing

Physical Basis

eROSITA was placed at the Sun–Earth L2 Lagrange point with its spin axis always pointing toward the Sun. As L2 orbits the Sun, the spin axis advances $\sim 1^\circ/\text{day}$ in ecliptic longitude, sweeping a great circle of constant ecliptic longitude each day. The scan rate is $0.025^\circ \text{ s}^{-1}$, completing one full rotation every 14,400 s (4 hours), so each sky position is crossed ~ 6 times per day.

Key assumptions:

- Scan starts at the South Ecliptic Pole ($\beta = -90^\circ$) at the beginning of each survey.
- Spin axis advances exactly $1^\circ/\text{day}$.
- Survey dates from Coutinho et al. (2022) — SRG/eROSITA: status and operations during the first four all-sky surveys, Proc. SPIE 12181, 2628946 (<https://doi.org/10.1117/12.2628946>) — stored in Cell 1.2.

Why Ecliptic Coordinates?

eROSITA’s scan geometry is defined by ecliptic longitude (λ) and latitude (β). A target at ecliptic longitude λ is scanned on the day the Sun reaches that same λ . A target at ecliptic latitude β is reached at a fixed time offset within each daily rotation:

$$t_{\text{from pole}} = \frac{\beta - (-90^\circ)}{0.025^\circ \text{ s}^{-1}}$$

Equatorial (RA, Dec) coordinates must therefore be converted to ecliptic (λ, β) before any timing calculation.

`get_sun_lambdas` (Cell 10)

For each eRASS survey, generates daily `Time` objects, calls `get_body('sun', times)` to retrieve the Sun’s sky position, and converts to `GeocentricMeanEcliptic` to extract λ_{sun} per day. Returns a list of `(survey_day_times, sun_lambda_all)` tuples. Called once; result is passed to all subsequent timing functions to avoid redundant computation.

`get_observation_time_eRosita` (Cell 11)

Takes equatorial (RA, Dec) and returns all eROSITA crossing times across all four surveys:

1. Convert (RA, Dec) $\rightarrow (\lambda_{\text{target}}, \beta_{\text{target}})$.
2. Find the survey day where $\lambda_{\text{sun}} \approx \lambda_{\text{target}}$ using the wraparound-safe angular difference: $|(\lambda_{\text{sun}} - \lambda_{\text{target}} + 180) \bmod 360 - 180|$.
3. Compute time offset from pole: $t = (\beta_{\text{target}} - (-90^\circ))/\text{scan_rate}$.
4. Add multiples of 14,400 s to get all crossings within the day.

`get_observation_time_eRosita_ec` (Cell 16.1)

Functionally identical to Cell 11 but accepts ecliptic (β, λ) directly, skipping the RA/Dec conversion. Used in batch timing (Cell 16.2) where coordinates are already in ecliptic form.

get_gw_time (Cell 12)

GW trigger times are retrieved in two ways:

- **New format (GWYYMMDD_HHMMSS):** UTC is parsed directly from the event name — no API call needed.
- **Old format (GWYYMMDD):** GWOSC API is queried with retry logic (3 attempts, exponential back-off: 1 s, 2 s, 4 s). GPS time (continuous seconds since 1980-01-06 00:00:00 UTC, no leap seconds) is converted to UTC via `Time(gps, format='gps')`.

Time Delay Computation

```
delay_s = (xray_time - gw_time).to(u.second).value
# Positive = eROSITA scanned after GW event (scientifically useful)
# Negative = scanned before (archival data predates the source)
```

Formatted as $\pm D d H h M m S s$ and written to CSV.

Fast Batch Timing (Cell 16.2)

Per-pixel timing for all overlap pixels of all top- N events is computationally expensive. Cell 16.2 computes timing only for the **center pixel** of each overlap region (median ecliptic longitude and corresponding latitude). The angular width of the overlap region in ecliptic longitude is also recorded (equivalent to the number of days eROSITA spent scanning through it). Use Cell 15.2 for precise per-pixel results for final publication figures.

Filtering (Cell 17)

Reads the timing CSV and keeps only records where `time_relative_to_gw_detection` begins with + (eROSITA scanned after the GW event). Output saved to `filtered_...csv`.

Validation: MAXI J1348–630 (Cell 19)

MAXI J1348–630 is a Galactic X-ray binary at ~ 3.8 kpc with a confirmed dust-scattering X-ray halo detected by eROSITA (Mastroserio et al. 2021). It is plotted on the scattering/absorption probability mask to confirm that the mask correctly includes sky regions where halos are known to be observable.

The halo is rendered as two **Annulus** patches:

- eRASS1: 34–40 arcmin radius
- eRASS2: 40–47 arcmin radius

The ring expands over time because X-ray photons scattered at larger angles travel longer path lengths and arrive progressively later, causing the echo to appear at growing angular radii. Note that MAXI J1348–630 is much closer than the BBH events and is not behind the full Galactic plane column density — it serves as a sanity check rather than a direct analog.

Graphing the SFD Dust Map (Optional, Cell 5)

Qualitative reference only — not used in any quantitative calculation. A 1400×700 equirectangular (RA, Dec) grid is built, `SFDQuery` is called at each point to retrieve $E(B-V)$ reddening, values are clipped between 0 and 2, inverted, and scaled to a grayscale image (low dust = white, high dust = dark gray). Displayed with `ax.imshow` and saved as PNG.